RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 7: Model Planning and Building - Clustering

Register Number: 241801187 **Date:** 12-10-2026

Dataset: Mall Customer Segmentation (Kaggle) **Tools:** Python, Pandas, Matplotlib, Seaborn, scikit-learn

Learning Objective

To understand and implement the K-Means clustering algorithm for customer segmentation.

Problem Statement

A retail company wants to group its customers by Annual Income and Spending Score to design personalised marketing campaigns. The task is to analyse the customer dataset, apply K-Means clustering to find customer segments, visualise the clusters and recommend suitable marketing strategies for each segment.

Task 1: Load the Mall Customer Segmentation dataset

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

sns.set_theme(style='whitegrid')
pd.set_option('display.width', 110); pd.set_option('display.max_columns', None)

df = pd.read_csv('Mall_Customers.csv')
print("Dataset loaded successfully")
```

Out [1]:

```
Dataset loaded successfully
```

> **Observation:** The Mall Customer Segmentation dataset is loaded into a DataFrame with 200 customers and 5 columns.

Task 2: Explore the dataset

In [2]:

```
print("First 5 records:"); print(df.head().to_string())
print("\nLast 5 records:"); print(df.tail().to_string())
print("\nShape:", df.shape)
print("\nData types:"); print(df.dtypes.to_string())
print("\nMissing values:"); print(df.isnull().sum().to_string())
print("\nDuplicate records:", df.duplicated().sum(), "| Duplicate CustomerID:", df['CustomerID'].duplicated().sum())
print("\nSummary statistics:"); print(df.describe().round(2).to_string())
print("\nGender counts:"); print(df['Gender'].value_counts().to_string())
```

Out [2]:

```
First 5 records:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Last 5 records:
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
```

```
196         197  Female   45                126                      28
197         198    Male   32                126                      74
198         199    Male   32                137                      18
199         200    Male   30                137                      83

Shape: (200, 5)

Data types:
CustomerID                int64
Gender                      str
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64

Missing values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0

Duplicate records: 0 | Duplicate CustomerID: 0

Summary statistics:
       CustomerID     Age  Annual Income (k$)  Spending Score (1-100)
count      200.00  200.00              200.00                  200.00
mean       100.50   38.85               60.56                   50.20
std         57.88   13.97               26.26                   25.82
min          1.00   18.00               15.00                    1.00
25%         50.75   28.75               41.50                   34.75
50%        100.50   36.00               61.50                   50.00
75%        150.25   49.00               78.00                   73.00
max        200.00   70.00              137.00                   99.00

Gender counts:
Gender
Female    112
Male       88
```

> **Observation:** There are 200 customers and 5 columns: CustomerID, Gender, Age, Annual Income (k$) and Spending Score (1-100). Gender is text and the rest are integers. There are no missing values and no duplicate records. The average customer is 38.85 years old, earns 60.56 k$ and has a spending score of 50.2. 112 customers are female and 88 are male.

Task 3: Select the features for clustering

In [3]:

```
features = ['Annual Income (k$)', 'Spending Score (1-100)']
X = df[features]
print("Selected features:", features)
print("Shape of X:", X.shape)
print(X.describe().round(2).to_string())
```

Out [3]:

```
Selected features: ['Annual Income (k$)', 'Spending Score (1-100)']
Shape of X: (200, 2)
       Annual Income (k$)  Spending Score (1-100)
count              200.00                  200.00
mean                60.56                   50.20
std                 26.26                   25.82
min                 15.00                    1.00
25%                 41.50                   34.75
50%                 61.50                   50.00
75%                 78.00                   73.00
max                137.00                   99.00
```

> **Observation:** Annual Income (k$) and Spending Score (1-100) are selected as the clustering features, as required. Income ranges from 15 to 137 k$ and spending score from 1 to 99.

Task 4: Standardise the features using StandardScaler

In [4]:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
scaled_df = pd.DataFrame(X_scaled, columns=features)
print("Before scaling:"); print(pd.DataFrame({'mean': X.mean(), 'std': X.std(ddof=0)}).round(2).to_string())
print("\nAfter scaling:"); print(pd.DataFrame({'mean': scaled_df.mean(), 'std': scaled_df.std(ddof=0)}).round(2).to_string())
print("\nFirst 5 standardised rows:"); print(scaled_df.head().round(4).to_string())
```

Out [4]:

```
Before scaling:
                         mean    std
```

```
Annual Income (k$)      60.56  26.20
Spending Score (1-100)  50.20  25.76

After scaling:
                        mean  std
Annual Income (k$)      -0.0  1.0
Spending Score (1-100)  -0.0  1.0

First 5 standardised rows:
   Annual Income (k$)  Spending Score (1-100)
0             -1.7390                -0.4348
1             -1.7390                 1.1957
2             -1.7008                -1.7159
3             -1.7008                 1.0404
4             -1.6627                -0.3960
```

> **Observation:** K-Means is based on distances, so the features must be on the same scale. After StandardScaler both features have mean 0 and standard deviation 1, and neither feature dominates the distance calculation. (StandardScaler divides by the population standard deviation, so the "before" values are shown with the same convention.)

Task 5: Visualise the selected features - scatter plot and pair plot

In [5]:

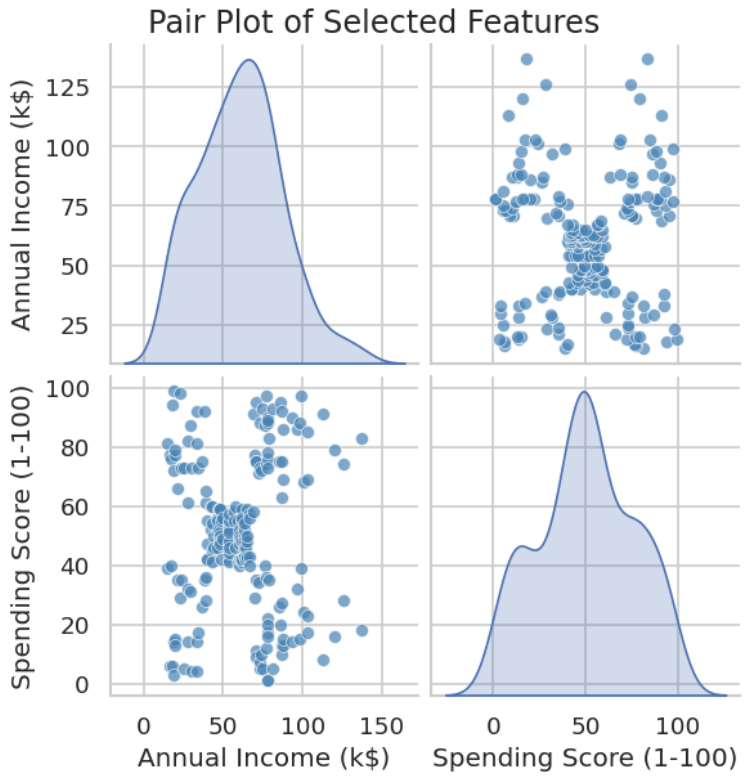
```
plt.figure(figsize=(7.5, 5.5))
plt.scatter(df[features[0]], df[features[1]], color='steelblue', alpha=0.7, edgecolor='white')
plt.title('Annual Income vs Spending Score'); plt.xlabel(features[0]); plt.ylabel(features[1])
plt.tight_layout(); plt.show()

g = sns.pairplot(df[features], diag_kind='kde', plot_kws={'alpha': 0.7, 'color': 'steelblue'})
g.fig.suptitle('Pair Plot of Selected Features', y=1.02); plt.show()
print("Correlation between Annual Income and Spending Score: %.3f" % df[features[0]].corr(df[features[1]]))
```

Out [5]:

```
Correlation between Annual Income and Spending Score: 0.010
```





Observation: The scatter plot shows clearly separated groups: a dense central group and four groups in the corners (low income with low or high spending, high income with low or high spending). The correlation between income and spending score is almost zero (0.010), so a straight-line relationship does not describe the customers, and clustering is a better way to find the segments. The pair plot shows that income is roughly bell-shaped and spending score has a main peak near 50.

Task 6: Determine the optimal number of clusters - Elbow Method

In [6]:

```
ks = range(1, 11)
inertia = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_scaled).inertia_ for k in ks]
sil = {k: silhouette_score(X_scaled, KMeans(n_clusters=k, random_state=42, n_init=10).fit_predict(X_scaled)) for k in range(2, 11)}
print("K  Inertia (WCSS)  Silhouette")
for k, w in zip(ks, inertia):
    print(f"{k:2d}  {w:12.2f}  {sil.get(k, float('nan')):10.4f}")

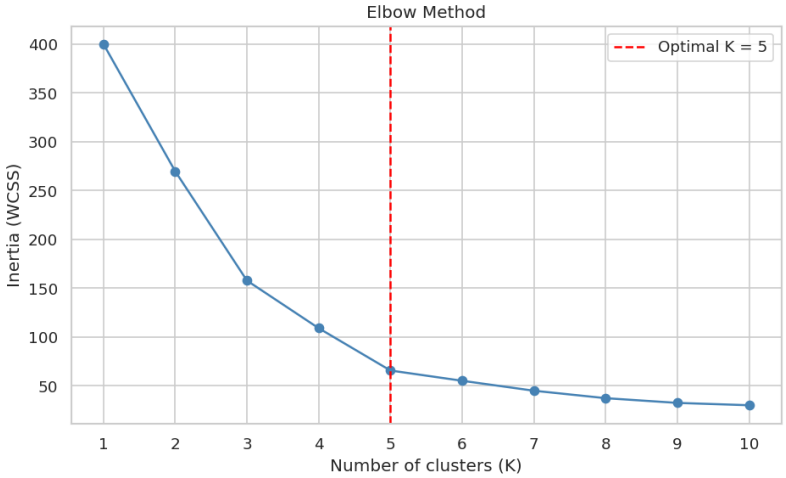
# elbow = point farthest from the straight line joining the first and last points of the curve
pts = np.column_stack([list(ks), inertia]).astype(float)
p1, p2 = pts[0], pts[-1]
dist = np.abs((p2[0] - p1[0]) * (p1[1] - pts[:, 1]) - (p1[0] - pts[:, 0]) * (p2[1] - p1[1])) / np.linalg.norm(p2 - p1)
optimal_k = int(list(ks)[dist.argmax()])
print("\nElbow (optimal K):", optimal_k)
print("K with the highest silhouette score:", max(sil, key=sil.get))

plt.figure(figsize=(8, 5))
plt.plot(list(ks), inertia, marker='o', color='steelblue')
plt.axvline(optimal_k, color='red', linestyle='--', label=f'Optimal K = {optimal_k}')
plt.title('Elbow Method'); plt.xlabel('Number of clusters (K)'); plt.ylabel('Inertia (WCSS)')
plt.xticks(list(ks)); plt.legend(); plt.tight_layout(); plt.show()
```

Out [6]:

```
K  Inertia (WCSS)  Silhouette
 1        400.00         nan
 2        269.69      0.3213
 3        157.70      0.4666
 4        108.92      0.4939
 5         65.57      0.5547
 6         55.06      0.5399
 7         44.86      0.5281
 8         37.23      0.4552
 9         32.39      0.4571
10         29.98      0.4432

Elbow (optimal K): 5
K with the highest silhouette score: 5
```



Observation: The inertia falls sharply up to K = 5 (65.57) and then flattens, so the elbow is at K = 5. The silhouette score is also highest at K = 5 (0.5547), so both methods agree. K = 5 is the optimal number of clusters.

Task 7: Train K-Means and save the cluster labels

In [7]:

```
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)
print("K-Means trained with K =", optimal_k, "| Inertia: %.2f" % kmeans.inertia_,
      "| Silhouette: %.4f" % silhouette_score(X_scaled, df['Cluster']))
df.to_csv('Mall_Customers_Clustered.csv', index=False)
check = pd.read_csv('Mall_Customers_Clustered.csv')
print("File saved: Mall_Customers_Clustered.csv | shape:", check.shape)
print(check.head(8).to_string())
```

Out [7]:

```
K-Means trained with K = 5 | Inertia: 65.57 | Silhouette: 0.5547
File saved: Mall_Customers_Clustered.csv | shape: (200, 6)
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  Cluster
0           1    Male   19                  15                      39        4
1           2    Male   21                  15                      81        2
2           3  Female   20                  16                       6        4
3           4  Female   23                  16                      77        2
4           5  Female   31                  17                      40        4
5           6  Female   22                  17                      76        2
6           7  Female   35                  18                       6        4
7           8  Female   23                  18                      94        2
```

Observation: K-Means with K = 5 gives an inertia of 65.57 and a silhouette score of 0.5547, which means the clusters are well separated. A Cluster column (0 to 4) was added to the dataset and saved as Mall_Customers_Clustered.csv (200 rows, 6 columns).

Task 8: Visualise the clusters and the cluster centroids

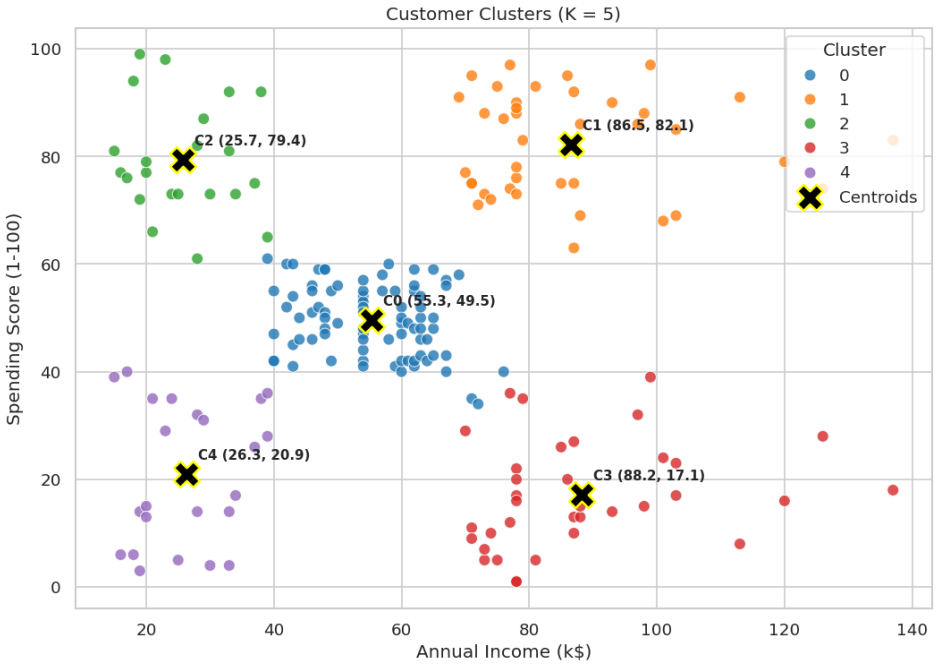
In [8]:

```
centroids = scaler.inverse_transform(kmeans.cluster_centers_)       # back to original units
centroid_df = pd.DataFrame(centroids, columns=features).round(2)
centroid_df.index.name = 'Cluster'
print("Cluster centroids (original units):"); print(centroid_df.to_string())

plt.figure(figsize=(9, 6.5))
sns.scatterplot(data=df, x=features[0], y=features[1], hue='Cluster', palette='tab10', s=60, alpha=0.8)
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=300, c='black', edgecolor='yellow', linewidth=1.5, label='Centroids')
for i, (x, y) in enumerate(centroids):
    plt.annotate(f"C{i} ({x:.1f}, {y:.1f})", (x, y), textcoords='offset points', xytext=(8, 10), fontsize=9, fontweight='bold')
plt.title(f'Customer Clusters (K = {optimal_k})'); plt.legend(title='Cluster', loc='upper right')
plt.tight_layout(); plt.show()
```

Out [8]:

```
Cluster centroids (original units):
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                     55.30                   49.52
1                     86.54                   82.13
2                     25.73                   79.36
3                     88.20                   17.11
4                     26.30                   20.91
```



Observation: The five clusters match the five visible groups. Centroids (income k$, spending score): C0 (55.3, 49.5), C1 (86.5, 82.1), C2 (25.7, 79.4), C3 (88.2, 17.1) and C4 (26.3, 20.9). The centroids were converted back from the scaled values to the original units so they can be read directly.

Task 9: Number of customers in each cluster

In [9]:

```
counts = df['Cluster'].value_counts().sort_index()
summary = pd.DataFrame({'Customers': counts, 'Percent': (counts / len(df) * 100).round(1)})
print(summary.to_string())
plt.figure(figsize=(7, 4.2))
ax = sns.barplot(x=counts.index, y=counts.values, hue=counts.index, palette='tab10', legend=False)
[ax.bar_label(c) for c in ax.containers]; plt.title('Customers per Cluster'); plt.xlabel('Cluster'); plt.ylabel('Number of Customers')
plt.tight_layout(); plt.show()
```

Out [9]:

```
         Customers  Percent
Cluster
0               81     40.5
1               39     19.5
2               22     11.0
3               35     17.5
4               23     11.5
```



Observation: Cluster 0 is the largest with 81 customers (40.5%), followed by Cluster 1 with 39 (19.5%), Cluster 3 with 35 (17.5%), Cluster 4 with 23 (11.5%) and Cluster 2 with 22 (11.0%). The average-income, average-spending group makes up about 40% of customers.

Task 10: Average Annual Income and Spending Score of each cluster

In [10]:

```
profile = df.groupby('Cluster')[features].mean().round(2)
profile['Average Age'] = df.groupby('Cluster')['Age'].mean().round(1)
profile['Customers'] = counts
print(profile.to_string())
print("\nGender split (%) in each cluster:")
print((pd.crosstab(df['Cluster'], df['Gender'], normalize='index') * 100).round(1).to_string())
```

Out [10]:

```
         Annual Income (k$)  Spending Score (1-100)  Average Age  Customers
Cluster
0                     55.30                   49.52         42.7         81
1                     86.54                   82.13         32.7         39
2                     25.73                   79.36         25.3         22
3                     88.20                   17.11         41.1         35
4                     26.30                   20.91         45.2         23

Gender split (%) in each cluster:
Gender   Female  Male
Cluster
0          59.3  40.7
1          53.8  46.2
2          59.1  40.9
3          45.7  54.3
4          60.9  39.1
```

Observation: Cluster 1 has high income (86.5 k$) and high spending (82.1) and a younger average age (32.7). Cluster 3 has high income (88.2 k$) but very low spending (17.1). Cluster 2 has low income (25.7 k$) but high spending (79.4) and is the youngest group (25.3 years). Cluster 4 has low income (26.3 k$) and low spending (20.9) and is the oldest (45.2 years). Cluster 0 is in the middle on both (55.3 k$, 49.5). Gender is fairly balanced in all clusters, with Cluster 3 the only one with more men (54.3%).

Marketing strategies for each segment

The problem statement asks for suitable marketing strategies. Based on the cluster profiles above, the following are suggested.

Cluster	Segment	Profile	Suggested strategy
1 (39)	High income, high spending	Income 86.5 k$, score 82.1, average age 32.7	Best customers: VIP loyalty programme, early access to new products, personalised premium offers and exclusive events to keep them.
3 (35)	High income, low spending	Income 88.2 k$, score 17.1, average age 41.1	Large unused potential: find out why they spend little (survey), then target them with quality and premium products, personalised recommendations and exclusive offers.
2 (22)	Low income, high spending	Income 25.7 k$, score 79.4, average age 25.3	Young, enthusiastic buyers: reward points, student or instalment offers, flash sales and social media campaigns, without deep discounts that cut margins.
4 (23)	Low income, low spending	Income 26.3 k$, score 20.9, average age 45.2	Price-sensitive: value bundles, discount coupons and seasonal sales on essentials, with low marketing spend.
0 (81)	Average income, average spending	Income 55.3 k$, score 49.5, average age 42.7	Largest group: cross-selling and bundle offers, loyalty points and seasonal campaigns to move them towards higher spending.

Observation: These strategies are suggestions based on the segment profiles only. The data has no information about product categories or profit, so the offers should be tested with small campaigns before a full rollout.

Result

Thus the Mall Customer Segmentation dataset (200 customers) was explored and the features Annual Income and Spending Score were standardised. The Elbow Method (and the silhouette score of 0.5547) gave K = 5 as the optimal number of clusters. K-Means with K = 5 produced five customer segments of 81, 39, 35, 23 and 22 customers, which were visualised with their centroids, and the cluster labels were saved in Mall_Customers_Clustered.csv. The segments are: average income and spending (Cluster 0), high income and high spending (Cluster 1), low income and high spending (Cluster 2), high income and low spending (Cluster 3) and low income and low spending (Cluster 4), and marketing strategies were suggested for each.